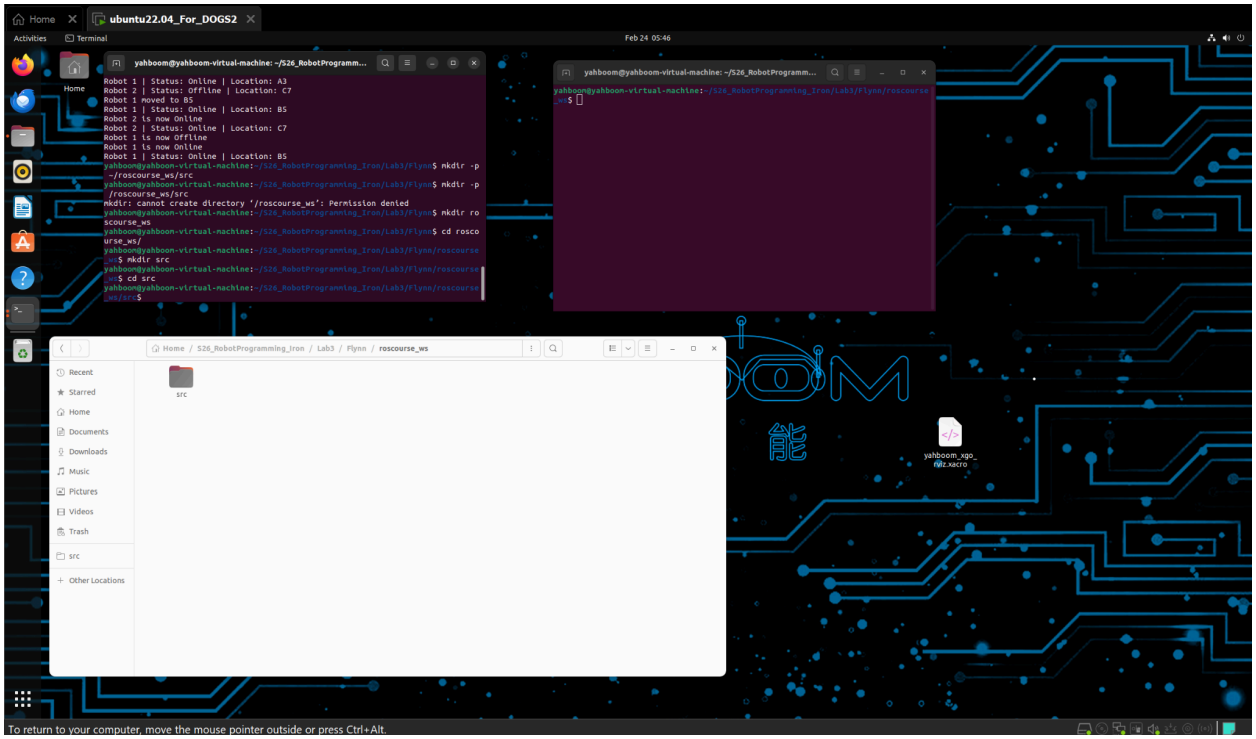
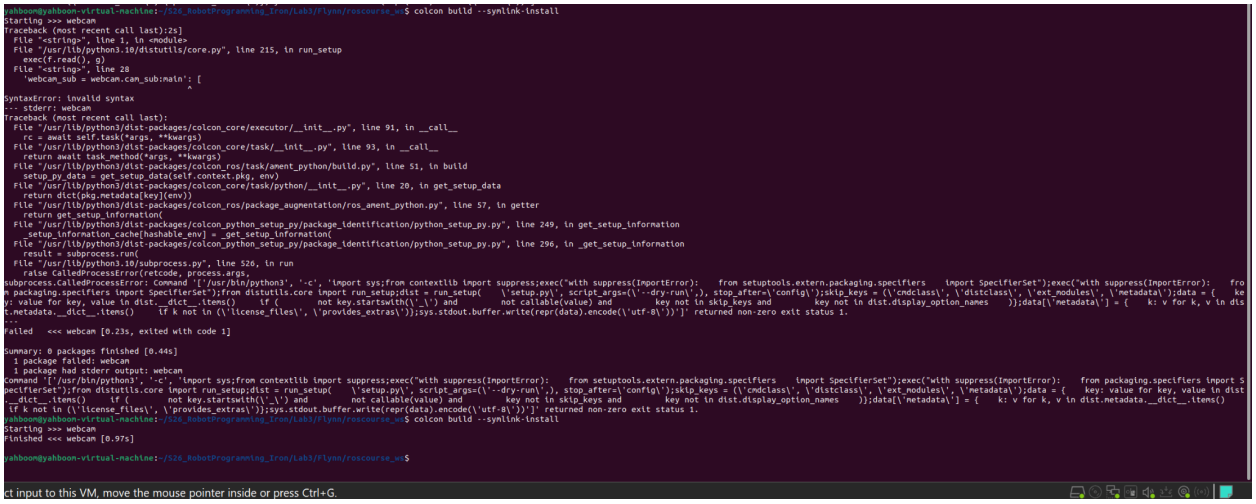
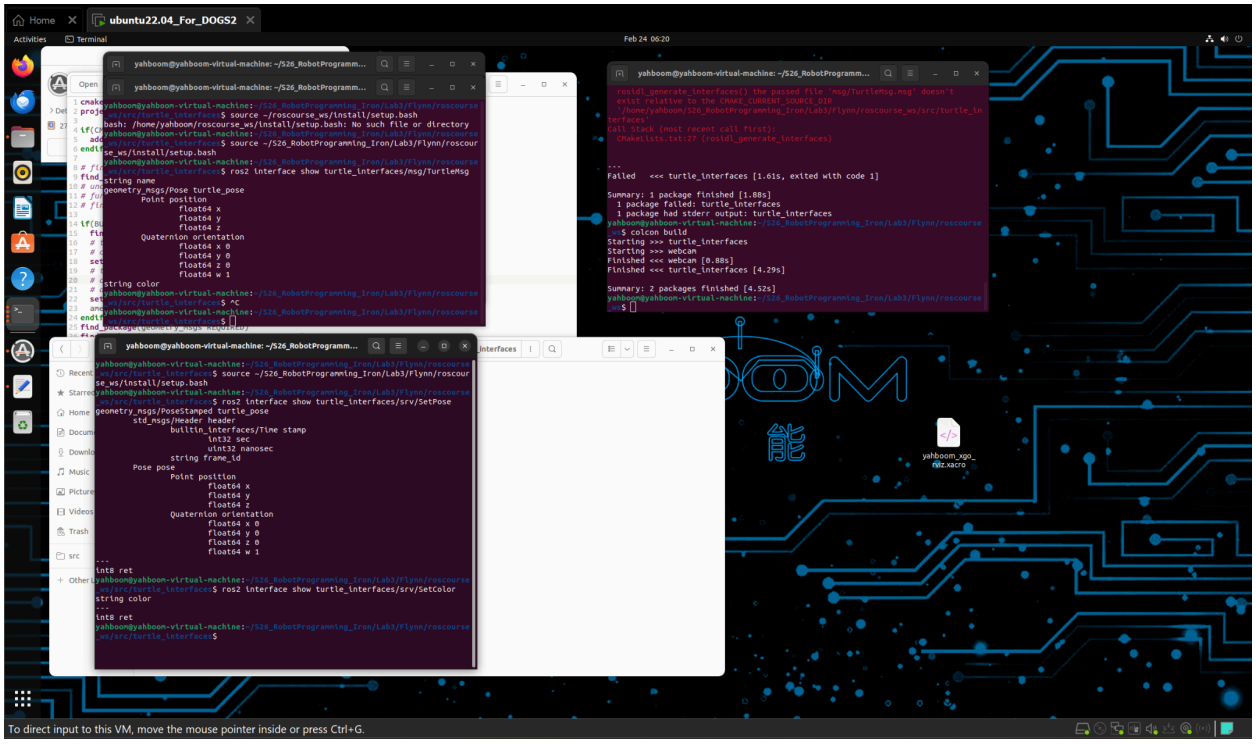
Lab 3 Task 1:



Task 2:



Task 3 step 22:



Task 4 required question step 3:

turtlebot_server.py

This file creates a ROS2 node called `turtleServer` that simulates a turtle robot's physics and manages its state. The key components are:

- `self.turtle = TurtleMsg()` — creates a turtle object using the custom message type we defined in `turtle_interfaces`, which stores the turtle's name, pose, and color
- `self.turtle_pub = self.create_publisher(TurtleMsg, 'turtleState', 1)` — publishes the turtle's current state (position, orientation, color) on the `turtleState` topic so other nodes can see it
- `self.twist_sub = self.create_subscription(Twist, 'turtleDrive', self.twist_callback, 1)` — subscribes to the `turtleDrive` topic to receive velocity commands, where `Twist` messages contain linear and angular velocity
- `self.create_timer(self.sim_interval, self.driving_timer_cb)` — calls `driving_timer_cb` every 0.02 seconds to update the turtle's position based on its current velocity

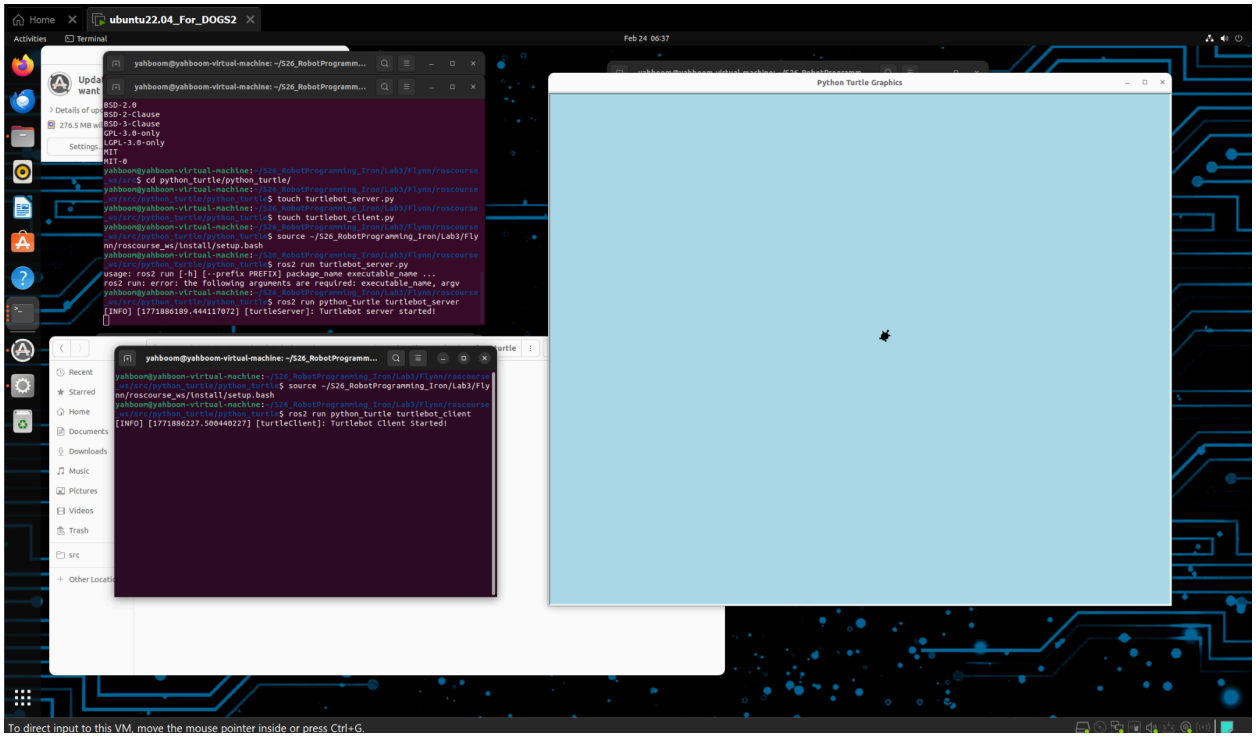
- `driving_timer_cb` — uses basic physics to calculate the turtle's new x/y position and yaw angle based on velocity, then converts the yaw to a quaternion for the pose message
- `set_color_callback` — handles service requests to change the turtle's color

turtlebot_client.py

This file creates a ROS2 node called `turtleClient` that controls the turtle and displays it visually using Python's `turtle` graphics library. The key components are:

- `self.screen` and `self.turtle_display` — sets up the Python turtle graphics window with a light blue background and a turtle shape to visualize the robot
- `self.twist_pub = self.create_publisher(Twist, 'turtleDrive', 1)` — publishes velocity commands to the `turtleDrive` topic which the server subscribes to
- `self.turtle_sub = self.create_subscription(TurtleMsg, 'turtleState', self.turtle_callback, 1)` — subscribes to `turtleState` to receive the turtle's current position and color from the server
- `turtle_callback` — updates the local turtle object whenever a new state message arrives from the server
- `update()` — uses the received state to update the visual turtle's position and heading on screen, and sets the pen color based on `self.turtle.color`
- In `main()`, `cmd_msg.linear.x = float(50 * unit_x)` and `cmd_msg.angular.z = float(1 * unit_z)` — these set the turtle's forward speed and turning rate. The `unit_x` and `unit_z` values (both set to 1) act as scaling ratios for the velocity c**ommands**

Task 4 step 7:



Task 4 step 16: